



Hands-On Lab Exercise 6: Loops in Shell Script

Topic: Repetition and Automation Using Loops (`for` , `while` , `until`)

Author: Dr. Sandeep Kumar Sharma

This lab explains how shell scripts **repeat tasks automatically** using loops.

Loops allow scripts to: - Repeat actions - Automate repetitive work - Process multiple items - Perform bulk operations - Build scalable automation

Without loops → scripts are limited.

With loops → scripts become **powerful automation engines**.



What are Loops?

Loops are used to:

Repeat the same task multiple times automatically.

In simple words:

"Do the same work again and again without writing the command again."



Why Do We Need Loops?

We need loops because: - Repetitive tasks exist everywhere - Automation must scale - Bulk operations are common - Systems have multiple files - Servers are multiple - Manual repetition is inefficient



Purpose of Loops

Loops help scripts to: - Process many files - Handle multiple users - Manage multiple servers - Automate bulk tasks - Save time - Reduce errors



Learning Objective

- Understand loop concepts
 - Learn loop types
 - Use `for`, `while`, `until`
 - Build repetitive automation
 - Write scalable scripts
-



Learning Outcome

After this lab, participants will: - Use loops in scripts - Automate repetitive tasks - Build bulk automation - Create scalable scripts - Think in automation logic



Types of Loops in Shell Script

1. `for` loop
 2. `while` loop
 3. `until` loop
-



Basic Syntax

`for` loop

```
for var in list
do
  commands
done
```

`while` loop

```
while [ condition ]
do
  commands
done
```

until loop

```
until [ condition ]  
do  
    commands  
done
```

Example 1: for Loop (Simple Counting)

Purpose:

Print numbers from 1 to 5

Step 1: Create Script

```
touch for1.sh
```

Step 2: Open File

```
nano for1.sh
```

Step 3: Write Script

```
#!/bin/bash  
  
for i in 1 2 3 4 5  
do  
    echo "Number: $i"  
done
```

Step 4: Permission

```
chmod +x for1.sh
```

Step 5: Run

```
./for1.sh
```

Example 2: `for` Loop (Directory Creation)

Purpose:

Create multiple directories automatically

```
#!/bin/bash

for dir in app logs config backup
do
    mkdir $dir
    echo "Directory $dir created"
done
```

Example 3: `while` Loop (Counting)

Purpose:

Print numbers from 1 to 5 using while loop

```
#!/bin/bash

i=1

while [ $i -le 5 ]
do
```

```
    echo "Number: $i"
    i=$(( i + 1 ))
done
```

Example 4: while Loop (File Creation)

Purpose:

Create multiple files automatically

```
#!/bin/bash

i=1

while [ $i -le 5 ]
do
    touch file$i.txt
    echo "file$i.txt created"
    i=$(( i + 1 ))
done
```

Example 5: until Loop (Counting)

Purpose:

Print numbers until condition becomes true

```
#!/bin/bash

i=1

until [ $i -gt 5 ]
do
    echo "Number: $i"
    i=$(( i + 1 ))
done
```



Example 6: Loop in Real Automation Scenario



Purpose:

Install package on multiple servers (simulation)

```
#!/bin/bash

for server in server1 server2 server3 server4
do
    echo "Connecting to $server"
    echo "Installing git on $server"
    echo "Installation completed on $server"
done
```

Understanding the Logic

- Loop repeats commands
- Same logic used multiple times
- Automation scales easily
- One script does bulk work

Key Understanding

- Loops = repetition engine
- Automation = repetition + logic
- Loops reduce manual work
- Loops build scalable systems



Summary

- Loops automate repetition
- `for`, `while`, `until` are core
- Loops save time
- Loops reduce errors
- Loops enable scalability

End of Lab

This lab teaches the **power of repetition in automation**.

Now scripts can: - Repeat tasks - Handle bulk operations - Scale automation - Manage large systems

This is core to: - DevOps automation - Cloud automation - Infrastructure automation - CI/CD systems

Author:

Dr. Sandeep Kumar Sharma

LinkedIn:

[linkedin.com/in/sandeep-kaushik-2a345856](https://www.linkedin.com/in/sandeep-kaushik-2a345856)

Medium Blog:

medium.com/@shyamsandeep28